

H-HPU

Plataforma de Aceleração Computacional Completa

SoC / Arquitetura de Chiplets com Stack de Software Dedicada

Whitepaper técnico conceitual - versão 0.3

Nota de escopo e estado do projeto

Este documento consolida, em uma única especificação, os elementos desenvolvidos ao longo do projeto H-HPU: matemática orientada a relações e geometria hiperbólica, malha hexagonal, H6-NoC, H-MEM, H-MESI, controlador HARC, ISA H6 e a stack de software correspondente. O texto trata a plataforma como uma arquitetura de pesquisa: as equações de topologia são derivadas, as métricas de referência vêm de literatura pública e os números de estresse do controlador são resultados de um modelo interno, não de silício fabricado.

O projeto adota co-design: a matemática define quais operações e relações aparecem com maior frequência; a estrutura de dados define como elas são armazenadas; a interconexão define como os dados se movem; o hardware expõe primitivas dedicadas; e o software fecha o ciclo com compilação, runtime, bibliotecas e controle adaptativo.

A palavra "completa" refere-se à abrangência da pilha descrita neste whitepaper, não a uma afirmação de tape-out, produção comercial ou superioridade universal frente a GPUs e outros aceleradores.

Resumo

Este whitepaper propõe a H-HPU, uma Plataforma de Aceleração Computacional Completa na qual algoritmo, representação de dados, topologia e hardware são co-projetados. O núcleo da proposta é uma rede de processamento e memória com conectividade hexagonal, acoplada a uma hierarquia de memória orientada à localidade, coerência direcional, roteamento adaptativo livre de impasses, um controlador de IA especializado e uma extensão de ISA para RISC-V. O objetivo é criar uma máquina na qual operações associadas a estruturas hierárquicas e kernels hiperbólicos deixem de ser abstrações de software e passem a ter suporte arquitetural direto.

A base matemática inclui o limite de planaridade $E \leq 3N - 6$, que implica grau médio assintótico ≤ 6 , e a análise de HexaMesh, que reporta redução assintótica de 42% no diâmetro e aumento de 130% na largura de bisseção frente a uma malha em grade, além de ganhos de latência e throughput em simulação de ciclo. [R1] O whitepaper usa essas relações como fundamento topológico, sem transferir automaticamente tais percentuais para desempenho de um H-HPU futuro.

No modelo interno já testado, o controlador HARC reduz a utilização de computação de 7,06x para 0,852x, a rede de 2,10x para 0,800x e a pressão de memória para 0,957x antes do overhead. Com 8% de overhead de memória, a pressão chega a 1,034x; a introdução do H-MEM com uma meta conservadora de 10% de redução de tráfego leva o valor a aproximadamente 0,930x. Esses números são usados como alvos de arquitetura e não como benchmark de hardware.

Palavras-chave

computação hiperbólica; geometria relacional; chiplets; Network-on-Chip; malha hexagonal; memória hierárquica; coerência; prefetch; IA para hardware; RISC-V; co-design; SoC; aceleradores.

Sumário

- 1. Motivação e problema de arquitetura
- 2. Princípios de projeto
- 3. Fundamentos matemáticos
- 4. Arquitetura H-HPU
- 5. H6-NoC: rede hexagonal
- 6. H-MEM: memória hierárquica
- 7. H-MESI: coerência direcional
- 8. HARC: IA de controle e correção
- 9. H6-ISA: conjunto de instruções
- 10. Stack de software dedicada
- 11. Modelo de programação
- 12. Modelo de desempenho e cálculos
- 13. Confiabilidade, segurança e observabilidade
- 14. Co-design físico e chiplets
- 15. Metodologia de validação
- 16. Roadmap de implementação
- 17. Limites e questões abertas
- 18. Conclusão
- Apêndice A. Notação
- Apêndice B. ISA de referência
- Apêndice C. Pseudocódigo
- Referências

1. Motivação e problema de arquitetura

As arquiteturas de IA contemporâneas são excelentes em um padrão específico de trabalho: operações regulares, vetorizadas e densas, com grandes fluxos de multiplicação-acumulação e acesso altamente estruturado à memória. Por exemplo, a documentação e os materiais técnicos atuais da NVIDIA descrevem Tensor Cores, tiled matrix multiplication e hierarquias de memória de alta largura de banda como elementos centrais das GPUs Blackwell. [R4]

O problema que motiva a H-HPU é diferente: quando a computação é dominada por relações entre entidades, hierarquias, vizinhanças e transformações geométricas, a organização dos dados e da comunicação pode tornar-se tão importante quanto a contagem de operações aritméticas. A proposta, portanto, não parte da pergunta "como executar uma operação hiperbólica em uma GPU", mas da pergunta mais abrangente "qual máquina minimiza o custo físico de uma família de algoritmos cuja estrutura é relacional e hierárquica?"

A crescente importância de memória e infraestrutura também é coerente com a evolução recente dos sistemas de IA. A IEA estima que os data centers consumiram cerca de 415 TWh em 2024, cerca de 1,5% da eletricidade mundial, e projeta cerca de 945 TWh em 2030 no caso-base. [R5] Pesquisas de 2026 sobre sistemas de memória orientados por aprendizado de máquina mostram que políticas adaptativas de prefetch, colocação e hierarquia podem superar heurísticas estáticas em determinados workloads. [R6]

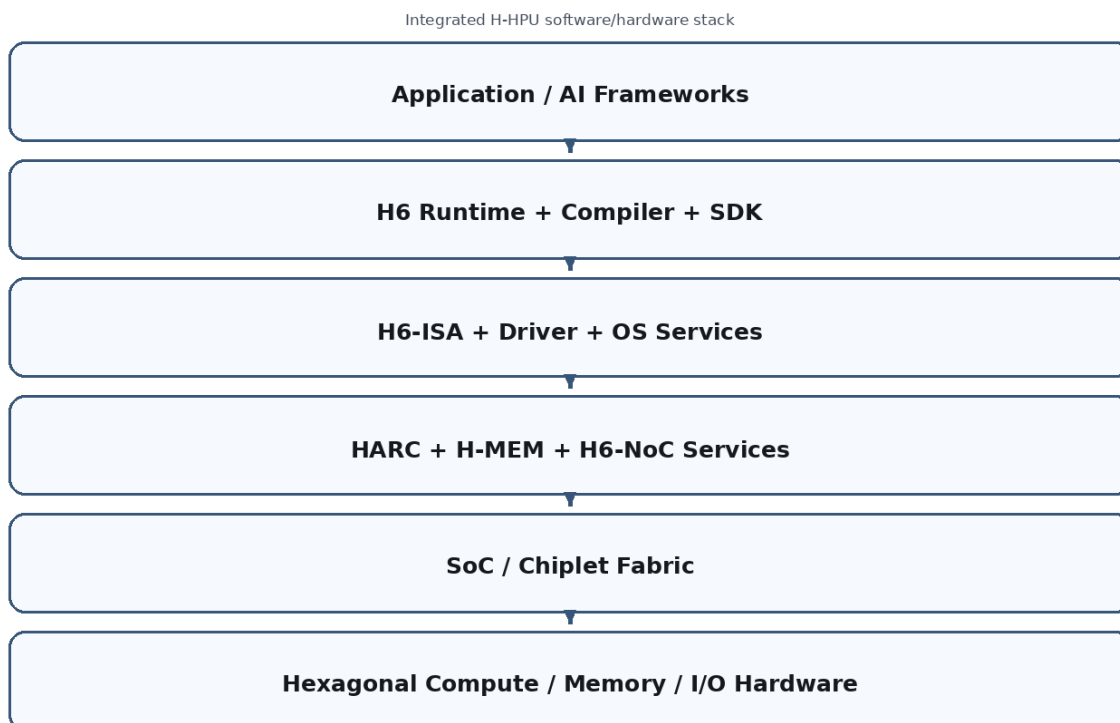


Figura 1 - Stack integrada da H-HPU: o software expõe intenções; o hardware explora relações e localidade.

1.1. Objetivo técnico

O objetivo do projeto é minimizar o tempo e a energia necessários para transformar uma estrutura de entrada em uma saída computacionalmente correta, sob uma arquitetura que favoreça relações locais e hierarquias. A função de custo de referência é:

$$L = \alpha * T + \beta * E + \gamma * M_{\text{move}} + \delta * A_{\text{chip}}$$

onde T é tempo, E é energia, Mmove é movimentação de dados e Achip é a área física estimada. Os coeficientes alpha, beta, gamma e delta representam prioridades de uma instância do produto.

1.2. O problema de co-design

$$P_{\text{total}} = P(\text{algoritmo, dados, interconexao, memoria, hardware, software})$$

Uma melhoria de complexidade não garante uma melhoria de desempenho se o hardware não puder explorar a estrutura que produz essa melhoria. O H-HPU trata o algoritmo e o hardware como componentes de uma mesma função de projeto.

2. Princípios de projeto

- Geometria como topologia: a conectividade matemática desejada deve ter um reflexo direto na interconexão.
- Relações como primeira classe: o sistema deve operar sobre relações locais e hierárquicas, e não apenas sobre matrizes densas.
- Memória próxima do cálculo: os dados que possuem alta probabilidade de reutilização devem permanecer próximos do núcleo que os consome.
- IA como controlador, não como árbitro absoluto: o agente pode otimizar rotas e políticas dentro de limites de segurança definidos por hardware.
- Falha localizada: erros, congestionamento ou degradação devem produzir uma resposta local sempre que possível.
- ISA mínima e composicional: instruções especializadas devem expressar intenção computacional, permitindo que o runtime e o compilador decidam os detalhes de execução.
- Compatibilidade: uma base RISC-V reduz o custo de construir um ecossistema de compiladores, ferramentas e sistemas operacionais.
- Observabilidade: todo mecanismo adaptativo deve gerar telemetria suficiente para ser auditado, reproduzido e desativado.

2.1. Princípio de preservação matemática

Quando o cálculo é definido em uma variedade, a representação de diferenças deve respeitar a geometria escolhida. A codificação inicial $x_i = x_{\text{parent}} + \Delta x_i$ é útil em uma coordenada Euclidiana, mas, para uma representação hiperbólica intrínseca, o análogo mais seguro é armazenar um vetor residual no espaço tangente:

$$v_i = \log_{\{x_p\}}(x_i), \quad x_i = \exp_{\{x_p\}}(v_i)$$

Esse mecanismo permite manter a ideia de compressão orientada à estrutura sem fingir que a subtração Euclidiana é invariavelmente equivalente à geometria hiperbólica.

3. Fundamentos matemáticos

3.1. Grafo de computação

$$G = (V, E)$$

Cada vértice v_i representa uma unidade lógica de processamento, memória ou roteamento; cada aresta representa um canal de comunicação permitido. A plataforma usa grafos planares em suas camadas hexagonais locais e pode usar links adicionais em níveis superiores quando a implementação deixar de ser estritamente planar.

3.2. Limite de conectividade planar

$$E \leq 3N - 6$$

$$d_{\text{medio}} = 2E/N \leq 6 - 12/N$$

No limite $N \rightarrow \infty$, o grau médio planar tende a no máximo 6. A interpretação arquitetural é que uma rede local plana pode explorar até seis relações médias por célula, e uma geometria honeycomb/hexagonal é uma forma natural de aproximar esse limite. HexaMesh explora essa propriedade para interconexão entre chiplets. [R1]

3.3. Grade versus HexaMesh

Para a referência em grade, o trabalho HexaMesh fornece, sob suas hipóteses, diâmetro $D_G = 2\sqrt{N} - 2$ e largura de bisseção $B_G = \sqrt{N}$. Para o arranjo HexaMesh, $D_H = (1/3)\sqrt{12N - 3} - 1/2$ e $B_H = (2/3)\sqrt{12N - 3}$. [R1]

$$\lim_{(N \rightarrow \infty)} D_H/D_G = 1/\sqrt{3} = 0,577350\dots$$

$$\text{Reducao assintotica} = 1 - 1/\sqrt{3} = 42,265\%$$

$$\lim_{(N \rightarrow \infty)} B_H/B_G = 4/\sqrt{3} = 2,309401\dots$$

$$\text{Ganho de bissecao} = 130,940\%$$

Esses resultados são propriedades do modelo topológico específico. Eles não significam que qualquer chip hexagonal será 42,3% mais rápido ou 130,9% mais largo em throughput.

N	D grade	D H-HPU/HM	Reducao D	B grade	B H-HPU/HM	Ganho B
100	18.000	11.033	38.71%	10.000	23.065	130.65%
1.000	61.246	36.010	41.20%	31.623	73.021	130.91%
10.000	198.000	114.969	41.94%	100.000	230.937	130.94%
100.000	630.456	364.648	42.16%	316.228	730.296	130.94%
1.000.000	1998.000	1154.200	42.23%	1000.000	2309.401	130.94%
10.000.000	6322.555	3650.984	42.25%	3162.278	7302.967	130.94%

Tabela 1 - Valores calculados a partir das fórmulas de HexaMesh. [R1]

3.4. Coordenadas axiais hexagonais

$$q, r \text{ in } \mathbb{Z}^2$$

$$\text{Direcoes} = \{(1,0), (1,-1), (0,-1), (-1,0), (-1,1), (0,1)\}$$

As coordenadas axiais permitem representar cada célula por dois inteiros e tornar a geração de vizinhos e a seleção de caminho explícitas. Em uma malha finita, as bordas reduzem o grau local; em uma arquitetura física é natural tratar o perímetro como região especial para I/O e distribuição de energia.

3.5. Operador relacional

$$F(v_i) = \sum_{\{v_j \text{ in } N(v_i)\}} w_{ij} * R(v_i, v_j)$$

$R(v_i, v_j)$ é a relação computacional entre células. O modelo pode representar distância, compatibilidade, similaridade, fluxo, dependência de dados ou qualquer relação que o algoritmo principal necessite. A função de peso w_{ij} incorpora prioridade, confiança, congestionamento e custo.

3.6. Complexidade alvo

$$Y(N) \sim O(N \log N)$$

O whitepaper trata $O(N \log N)$ como meta de complexidade para a classe de algoritmos em estudo. A arquitetura não pressupõe que todo kernel hiperbólico seja $O(N \log N)$; o valor só entra no modelo quando o algoritmo específico demonstrar essa redução.

4. Arquitetura H-HPU

A H-HPU é organizada em dois níveis complementares: uma camada de chiplets, destinada a escalar capacidade e manufatura, e uma camada interna de unidades hexagonais, destinada a organizar computação, memória local e roteamento. Essa separação permite que chiplets físicos permaneçam retangulares mesmo quando a topologia lógica é hexagonal, uma abordagem compatível com a motivação de HexaMesh e com restrições de fabricação. [R1]

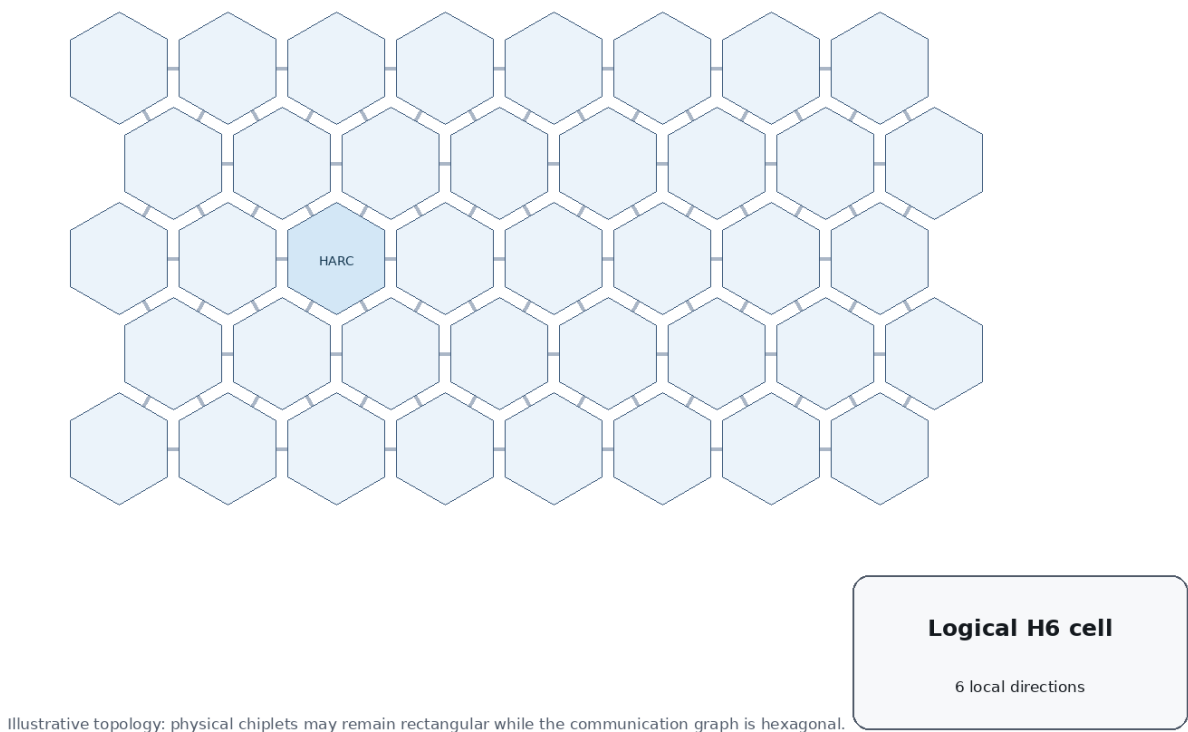


Figura 2 - Topologia H6 lógica. O chiplet físico pode permanecer retangular; a conectividade segue um grafo hexagonal.

4.1. Blocos de um chiplet

Bloco	Responsabilidade	Interface
HCU	unidades de cálculo relacional/hiperbólico	H6-GEO, SRAM local
H-L0/L1	registradores, scratchpad e cache local	HLOAD/HSTORE
H6-Router	roteamento de seis direções	HROUTE/VC
H-MESI Directory	estado de coerência e versão	HACQ/HREL/HINV
HARC Agent	predição e controle adaptativo	telemetria/atuidores
H6-IO	I/O, NIC/host, links de pacote	PCIe/CXL/SerDes/PHY
Safety Fabric	watchdog, limites, rollback	CSR/interrupt

4.2. Modelo SoC

Em uma implementação SoC, o host RISC-V fornece execução geral e supervisão. A malha H6 funciona como domínio acelerado. Em uma implementação chiplet, múltiplos chiplets H-HPU compartilham uma camada de interconexão de pacote, memória de alta largura de banda e I/O periférico.

4.3. Separação data plane / control plane

O data plane carrega dados e operações frequentes. O control plane carrega telemetria, políticas, invalidações, sinais de erro e comandos de reconfiguração. A IA HARC opera majoritariamente no control plane, evitando colocar inferência pesada no caminho crítico de cada flit.

5. H6-NoC: rede hexagonal

A H6-NoC é uma rede de comunicação de seis direções. Seu roteador combina um mecanismo determinístico de legalidade com uma política adaptativa. A regra de legalidade define quais turnos são permitidos; a política escolhe entre caminhos legalmente admissíveis com base em congestionamento, energia e falhas.

H6-NoC: six directions with constrained turns

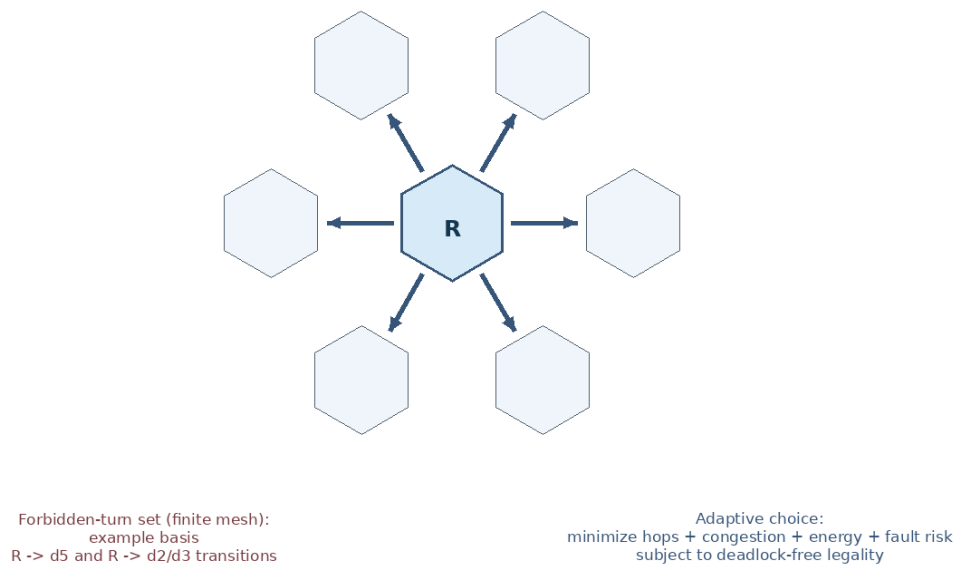


Figura 3 - Princípio de roteamento H6: adaptatividade sob um conjunto de turnos permitido.

5.1. Roteamento por caminhos mínimos

$$P_{\min}(s,d) = \{p : \text{len}(p) = \text{dist}(s,d)\}$$

$$p^* = \text{argmin}_p [\lambda_h H(p) + \lambda_q Q(p) + \lambda_e E(p) + \lambda_f F(p)]$$

A seleção é restrita a $P_{\min}$ ou a um conjunto de desvios controlados quando um link falha. O objetivo é evitar que a IA sacrifique previsibilidade em troca de uma otimização não verificável.

5.2. Deadlock e dependência de recursos

Deadlock em wormhole routing pode ser modelado por ciclos no channel dependency graph (CDG). A regra de projeto é eliminar ciclos do CDG por restrições de turnos e, quando necessário, por virtual channels. Uma pré-publicação de setembro de 2026 demonstra uma construção para malha triangular de grau seis: uma malha finita usa uma relação de roteamento parcialmente adaptativa com uma VC e apenas dois turnos direcionais proibidos; a versão toroidal usa duas VCs e datelines específicos para eliminar a dependência periódica. [R2]

CDG = (C, A); ciclo em CDG => possibilidade de deadlock

regra H6: construir CDG acíclico para as rotas admissíveis

No H-HPU v0.3, o conjunto exato de turnos proibidos é parametrizado pelo orientador da implementação. Isso permite aproveitar a construção estudada por [R2] sem amarrar o chip a uma única convenção de orientação.

5.3. Tabela de legalidade

Elemento	Função	Garantia
Turn table	proíbe turnos que fechariam ciclos	segurança topológica
VC 0	requests / data forward	separação de dependência
VC 1	responses / ACK / control	evita retorno circular
Fault mode	desvio triangular / rotação de regra	continuidade local
HARC policy	escolha entre rotas permitidas	adaptação de tráfego

5.4. Falha de enlace

Para uma falha bidirecional estática conhecida antes de uma época de roteamento, a construção de [R2] usa uma regra orientada para a direção da falha e substitui o salto quebrado por um bypass triangular de dois hops, preservando conectividade e adicionando no máximo um hop à distância mínima saudável.

6. H-MEM: memória hierárquica orientada à geometria

H-MEM é a memória em quatro níveis proposta para transformar a localidade matemática em localidade física. O objetivo não é apenas aumentar capacidade, mas reduzir movimentação e manter dados relevantes próximos das unidades que os consomem.

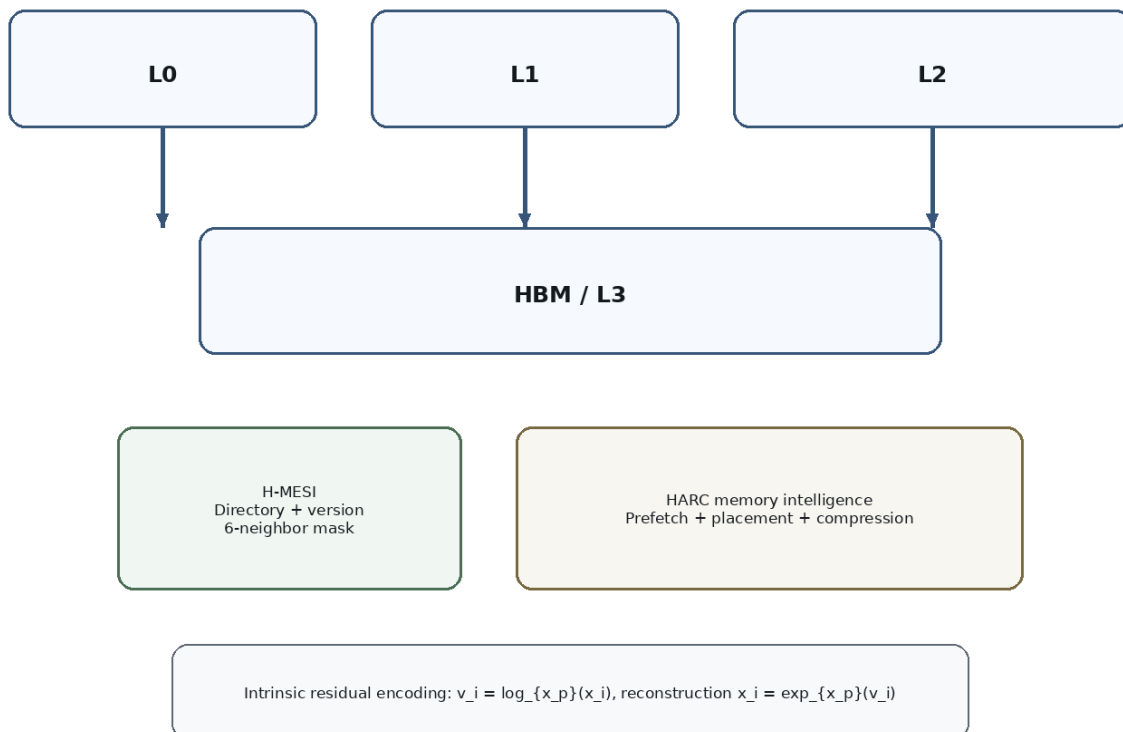


Figura 4 - H-MEM: hierarquia, coerência e inteligência de prefetch/placement.

6.1. Hierarquia

Nível	Tipo	Objetivo	Política
L0	registradores / scratchpad	dados imediatos	programado
L1	cache local	reuso intranó	HARC
L2	memória de cluster	reuso entre vizinhos	HARC + directory
L3	HBM / memória externa	capacidade global	prefetch / streaming

6.2. Localidade hexagonal

$$N(H_i) = \{H_1, \dots, H_6\}$$

Dados com alta probabilidade de transferência entre vizinhos são colocados em regiões de memória que minimizam a distância física. Isso cria uma função de custo de colocação:

$$C_{\text{place}}(x) = \sum_j P(\text{access}_j | x) * d_{\text{mem}}(x, j)$$

O HARC tenta minimizar C_{place} sem violar capacidade, coerência e confiabilidade.

6.3. Compressão orientada à estrutura

A forma simples $x_i = x_{\text{parent}} + \Delta x_i$ é preservada como modelo para dados Euclidianos. Para dados hiperbólicos, a forma intrínseca proposta é:

$$v_i = \log_{\{x_p\}}(x_i)$$

$$x_i = \exp_{\{x_p\}}(v_i)$$

O armazenamento pode guardar x_p uma vez por bloco e um conjunto de vetores residuais v_i . O ganho real de bits depende da estatística do conjunto e precisa ser medido por workload.

6.4. Prefetch orientado por IA

$$P_i(t+\tau) = \Pr(\text{access } i \text{ at } t+\tau | \text{state}_t)$$

O prefetch só é disparado quando a probabilidade estimada supera um limiar e quando existe margem de banda. Pesquisas de 2026 sobre sistemas de memória orientados por aprendizado apresentam prefetch e placement aprendidos como mecanismos viáveis para substituir políticas estáticas em workloads selecionados. [R6]

6.5. Resultado de estresse do projeto

No modelo interno utilizado na iteração atual, a pressão de memória após HARC ficou em 0,957 antes do overhead. Introduzindo 8% de overhead de memória, obtivemos:

$$U_M = 0,957 * 1,08 = 1,03356$$

Para voltar a $U_M < 1$, a redução mínima no tráfego efetivo é:

$$r_{\text{min}} = 1 - 1/1,03356 = 0,0325 = 3,25\%$$

Com uma meta conservadora de 10% de redução de tráfego:

$$U_{M_{\text{prime}}} = 1,03356 * (1 - 0,10) = 0,93020$$

Esse resultado fecha o gargalo de memória no modelo de controle usado para esta versão. Não é um benchmark de uma SRAM, HBM ou chip físico.

7. H-MESI: coerência direcional

A coerência H-MESI adapta a ideia de estados de cache para uma rede cuja vizinhança imediata contém até seis nós. Cada linha possui estado, proprietário, máscara de sharers locais, resumo de clusters e versão. O diretório é hierárquico para evitar broadcast global.

Campo	Descrição
state	I/S/E/M + T transitório
owner	nó proprietário quando exclusivo
S6	máscara de até seis vizinhos
SC	conjunto/resumo de clusters remotos
version	versão monotônica da linha
epoch	época de coerência/controle

7.1. Caminho de escrita

READ_SHARED -> INVALIDATE -> ACK -> M -> WRITE(version+1)

O nó solicitante começa com HACQ. O diretório identifica os sharers reais. Invalidações são enviadas apenas a quem possui a cópia. Depois de ACKs válidos, o novo proprietário recebe exclusividade. A versão da linha impede que uma cópia antiga seja confundida com a atual.

7.2. Coerência local e remota

No nível local, o diretório usa uma máscara de 6 bits. Em nível remoto, usa resumo por cluster. Essa combinação preserva o caminho curto da geometria local sem exigir que cada linha mantenha uma lista global de todos os consumidores.

7.3. Interação com HARC

HARC pode antecipar hotspot de coerência. Se um tensor começa a ser escrito frequentemente por nós distintos, o agente pode migrar a afinidade do tensor para um cluster, alterar política de replicação ou reduzir agressividade de prefetch para evitar cópias que logo serão invalidadas.

8. HARC: IA de controle e correção

HARC (Hyperbolic Adaptive Resource & Correction Controller) é a camada de controle adaptativo da plataforma. Ela não substitui ECC, roteamento determinístico ou verificadores de segurança; atua acima deles, medindo o estado e escolhendo políticas.

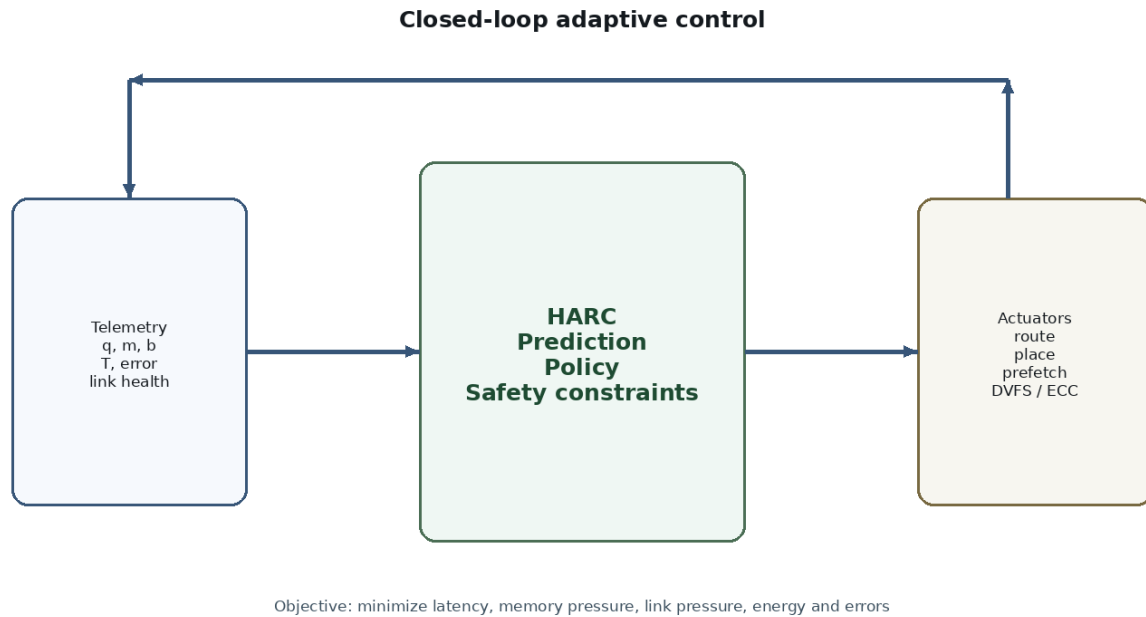


Figura 5 - HARC como malha fechada: mede, prediz, decide e atua.

8.1. Estado

$$s_i(t) = [q_i, m_i, b_i, T_i, e_i, l_i, f_i]$$

Os componentes representam fila, pressão de memória, banda, temperatura, taxa de erro, saúde do enlace e estado de falha.

8.2. Ação

$$a_i = [\text{route, placement, prefetch, ECC, DVFS, migration}]$$

A política pode ser um modelo leve embarcado, como uma rede pequena, árvore de decisão, perceptron ou agente de reinforcement learning. A escolha do modelo não é fixada pela ISA; o runtime pode carregar modelos diferentes desde que respeitem o contrato de segurança.

8.3. Função objetivo

$$J = wT \cdot T + wM \cdot M^2 + wN \cdot N^2 + wC \cdot C^2 + wE \cdot E + wF \cdot F$$

A forma quadrática penaliza estados próximos à saturação. Na implementação, os pesos podem ser ajustados por workload. Uma restrição dura mantém os atuadores dentro dos limites elétricos e térmicos do pacote.

8.4. Correção de gargalos

Gargalo	Sinal	Resposta HARC	Limite físico
Congestionamento	Q alta	migrar/rotear	capacidade dos links
Memória	BW alta	prefetch/placement/compressão	bandwidth/capacidade
Carga desigual	$U_i \gg U_{med}$	redistribuir	dependências
Link degradado	BER/CRC	re-rotear	conectividade
Erro de bit	ECC events	fortalecer proteção/migrar	integridade
Temperatura	T alta	DVFS/migrar	limite térmico

8.5. Resultado interno do controlador

Recurso	Sem HARC	Com HARC	Interpretação
Computação	7,06x	0,852x	abaixo da saturação
Memória	7,25x	0,957x	perto da saturação
Rede	2,10x	0,800x	folga de rede

O modelo indicou que a memória se tornava o limitante residual. O H-MEM foi então adicionado ao loop, reduzindo a pressão para 0,930x no cenário de 10% de redução de tráfego efetivo.

8.6. Segurança do controlador

HARC nunca deve ter autoridade para violar uma política de segurança ou uma restrição de integridade. As decisões ficam enquadradas por: (i) tabela de turnos legal; (ii) verificador de endereços; (iii) limites térmicos; (iv) estados de coerência; (v) watchdog; (vi) caminho de fallback determinístico.

9. H6-ISA: conjunto de instruções

A base recomendada é RV64G ou um perfil de servidor RISC-V equivalente, estendido com instruções H6 em espaços de opcode custom. A especificação RISC-V de janeiro de 2026 mantém custom-0 a custom-3 como espaços destinados a extensões customizadas. [R3]

H6 custom ISA layered on an open base ISA

Base ISA	RV64G / OS / exceptions
H6-MEM	HLOAD HSTORE HPREF HENC HDEC
H6-NOC	HROUTE HWAIT HFAULT
H6-COH	HACQ HREL HINV HACK
H6-GEO	HDIST HLOGM HEXPM HINNER

Conceptual opcode allocation uses RISC-V custom spaces; exact encoding is a v0.3 design target, not a ratified standard.

Figura 6 - ISA H6 proposta sobre uma base RISC-V.

9.1. Classes de instruções

Classe	Exemplos	Efeito arquitetural
Memória	HLOAD/HSTORE	movimento entre níveis H-MEM
Compressão	HENC/HDEC	residual Euclidiano ou hiperbólico
Prefetch	HPREF	inicia predição/transferência
Roteamento	HROUTE/HWAIT	envia e sincroniza tiles
Coerência	HACQ/HREL/HINV/HACK	propriedade e invalidação

Classe	Exemplos	Efeito arquitetural
Geometria	HDIST/HINNER/HLOGM/HEXPM	primitivas hiperbólicas
Confiabilidade	HFAULT/HCRC	leitura de estado e correção

9.2. Semântica de referência

As instruções abaixo são uma especificação funcional de v0.3, não uma codificação binária ratificada.

HLOAD rd, desc: carrega tile descrito por desc para o domínio local

HSTORE rs, desc: persiste tile a partir do domínio local

HPREF rs1, rs2, imm: registra stream e horizonte para o controlador de prefetch

HENC rd, parent, delta: gera residual estruturado

HDEC rd, parent, delta: reconstrói o estado do filho

HROUTE rs1, qd, rd: emite pacote/tile para coordenada destino

HWAIT rs1: espera conclusão de transação H6

HACQ addr: solicita propriedade exclusiva de uma linha

HREL addr: libera propriedade

HINV addr: emite invalidação autorizada pelo diretório

HACK tx: confirma ACK de coerência

HDIST rd, x, y: calcula distância hiperbólica conforme formato configurado

HLOGM rd, base, x: loga mapa em espaço tangente

HEXPM rd, base, v: exponencia mapa em espaço tangente

HINNER rd, x, y: produto interno no modelo geométrico escolhido

HFAULT rd: lê estado de falha/telemetria

HCRC rd, block: dispara/checks de integridade

9.3. Codificação proposta

Uma opção é distribuir as instruções entre custom-0 e custom-3 conforme domínio: custom-0 para memória, custom-1 para NoC/coerência, custom-2 para geometria e custom-3 para telemetria/controle. A codificação final deve considerar conflitos com futuras extensões locais e o sistema de toolchain.

10. Stack de software dedicada

A arquitetura só se torna utilizável quando a intenção do algoritmo puder chegar ao hardware sem o programador conhecer registradores internos. A stack é composta por cinco camadas: linguagem e bibliotecas, compilador, runtime, driver/OS e monitor HARC.

Camada	Componentes propostos	Papel
Application	Python/C++/Rust	uso do acelerador
Libraries	H6Tensor, H6Graph, H6Geom	operações e kernels
Compiler	LLVM/MLIR backend	lowering para H6-ISA
Runtime	H6RT	fila, memória, sincronização

Camada	Componentes propostos	Papel
Driver	H6DRV	mapeamento, interrupções, telemetria
OS services	IOMMU, isolamento, power mgmt	segurança e recursos
Control	HARC model manager	políticas adaptativas

10.1. Biblioteca H6Geom

A biblioteca deve fornecer abstrações para bola de Poincare, modelo de Lorentz e outras representações hiperbólicas, com conversões explícitas. Operações que possuem suporte H6 são marcadas no IR para lowering direto.

10.2. Compilação

O compilador transforma um grafo de operações em três planos: cálculo, movimento de dados e controle. Operações que compartilham uma região H6 recebem placement hints; sequências de acesso tornam-se descritores de prefetch; operações de coerência explícita tornam-se primitivas do runtime.

10.3. Runtime

H6RT mantém: filas por cluster, allocators conscientes da topologia, tabelas de streams, pools de memória, handles de transação e métricas. A API não expõe diretamente o roteamento de baixo nível, salvo em modo de pesquisa.

10.4. Monitor e profiler

O profiler deve registrar ciclos por componente, hits de L0/L1/L2, bytes movidos, hops por mensagem, ocupação de VCs, invalidações, erro corrigido e ações HARC. Esses dados alimentam o simulador e os modelos de otimização.

11. Modelo de programação

O programador trabalha com três abstrações: tensor, relação e região. Tensor é o dado; relação é a dependência entre itens; região é o conjunto de células H6 nas quais o trabalho reside.

11.1. Tensor relacional

$$R(X,Y) \rightarrow Z$$

O runtime tenta mapear X e Y para regiões próximas. Um kernel hiperbólico pode declarar a necessidade de distância, log-map, exp-map e agregação local sem controlar diretamente os links.

11.2. Região e afinidade

$$\text{Affinity}(X) = \text{argmin_region } C_place(X)$$

A afinidade é reavaliada quando o padrão muda. HARC pode promover, migrar ou replicar dados quentes.

11.3. Exemplo de pseudocódigo de alto nível

```

```python
with h6.region("embedding") as R:
 X = h6.tensor(data, geometry="lorentz")
 h6.prefetch(X, horizon=8)
 Y = h6.hyperbolic_distance(X, centroids)
 Z = h6.reduce_local(Y)
 h6.commit(Z)
```

```

O compilador identifica os kernels e emite H6-ISA sem expor a tabela de turnos ou os detalhes de H-MESI ao usuário.

12. Modelo de desempenho e cálculos

A avaliação precisa separar custo computacional de custo de dados. Para uma execução fortemente sobreposta:

$$T_{\text{iter}} \approx \max(T_{\text{compute}}, T_{\text{memory}}, T_{\text{NoC}}, T_{\text{coherence}}, T_{\text{control}})$$

Quando os componentes são serializados, usa-se soma em vez de máximo. O whitepaper recomenda reportar ambos os modelos.

12.1. Tempo de computação

$$T_{\text{compute}} = \text{Ops} / \text{Theta}_{\text{compute}}$$

12.2. Memória

$$T_{\text{memory}} = \text{Bytes} / \text{BW}_{\text{effective}}$$

$$\text{BW}_{\text{effective}} = \text{BW}_{\text{peak}} * \text{locality}_{\text{factor}} * \text{compression}_{\text{factor}} * \text{placement}_{\text{factor}}$$

12.3. Comunicação

$$T_{\text{NoC}} = H_{\text{avg}} * t_{\text{hop}} + T_{\text{queue}} + T_{\text{route}}$$

Como o diâmetro hexagonal de referência escala como aproximadamente 0,57735 do da grade, a parcela de tempo diretamente proporcional ao número de hops pode cair na mesma proporção em um caso ideal onde a largura de enlace e o restante do sistema sejam iguais. A literatura de HexaMesh, porém, encontrou 19% de redução média de latência em seus próprios experimentos, o que ilustra por que não se deve converter automaticamente geometria em desempenho de sistema. [R1]

12.4. Coerência

$$T_{\text{coherence}} \approx T_{\text{req}} + T_{\text{inv}} + T_{\text{ack}} + T_{\text{commit}}$$

A meta do H-MESI é manter T_{inv} limitado ao conjunto real de sharers e resolver o caso local em um raio de poucos hops.

12.5. Cálculo integrado de saturação

Para um recurso com demanda λ e capacidade C :

$$U = \lambda / C$$

O projeto busca $U < 1$. Para a memória:

$$U_{\text{M,base}} = 0,957$$

$$U_{\text{M,overhead}} = 0,957 * 1,08 = 1,03356$$

$$r_{\text{min}} = 1 - 1/1,03356 = 3,25\%$$

$$U_{\text{M,10\%}} = 1,03356 * 0,90 = 0,93020$$

Portanto, uma redução de apenas 3,25% do tráfego efetivo já basta no modelo para cruzar o limiar de não saturação; 10% fornece margem adicional de aproximadamente 6,98 pontos percentuais.

12.6. Métrica geométrica nova

$$X_i = B / D$$

$$X_{\text{grade}} \rightarrow 0,5$$

$$X_{\text{hex}} \rightarrow 2$$

$$Xi_hex / Xi_grade \rightarrow 4$$

Xi é uma métrica proposta neste projeto para capturar simultaneamente capacidade de bisseção e diâmetro. Não é uma métrica padrão da literatura; deve ser validada contra throughput, energia e área em simulações futuras.

13. Confiabilidade, segurança e observabilidade

13.1. ECC em camadas

O sistema usa ECC físico nas memórias e links críticos. HARC decide quando aumentar proteção, migrar uma página, retirar um link do uso ou marcar um chiplet como degradado. A IA não é responsável por corrigir bits silenciosos sem verificação.

13.2. Watchdog

Cada agente HARC possui timeout e heartbeat. Se o agente não responde, a malha retorna a uma política determinística. O rollback deve preservar coerência, integridade de dados e limites térmicos.

13.3. Isolamento

O H6RT deve respeitar IOMMU e mecanismos de memória do sistema. Uma aplicação não deve conseguir emitir HROUTE para região que não possui permissão.

13.4. Observabilidade

| Métrica | Granularidade | Uso |
|------------------|------------------|------------------------------|
| Hops/flit | por transação | otimização de rota |
| VC occupancy | por roteador | detecção de congestionamento |
| Cache hit | por nível | ajuste de placement/prefetch |
| Coherence events | por cluster | controle de replicação |
| ECC/CRC | por link/memória | confiabilidade |
| Temperature | por chiplet | DVFS/migração |
| HARC action log | por época | auditoria/reprodução |

14. Co-design físico e chiplets

A arquitetura é projetada para aproveitar chiplets sem exigir que cada die tenha formato hexagonal. HexaMesh mostrou que a forma e o arranjo dos chiplets podem mudar significativamente o desempenho da interconexão e que chiplets retangulares podem organizar uma topologia honeycomb/HexaMesh. [R1]

14.1. Layers físicas

| Nível | Elemento | Objetivo |
|----------------|---------------------|------------------------|
| Die | HCU + SRAM + Router | computação local |
| Package | chiplet links | banda de curto alcance |
| Memory package | HBM/L3 | capacidade e banda |
| Board | I/O / NIC | escala do sistema |
| Rack | hosts H-HPU | escala de cluster |

14.2. Links e PHY

Cada link H6 precisa de buffers, codificação de integridade, CRC e potencial correção. A literatura de chiplets destaca que D2D link bandwidth e seu comprimento são fatores críticos na interconexão 2.5D. [R1] Portanto, a topologia deve ser combinada com um orçamento físico de comprimento, bump pitch, sinalização e potência.

14.3. Energia e termal

A HARC pode aplicar DVFS por cluster, mas a primeira linha de defesa é a distribuição física de trabalho. A meta não é "esfriar" uma região com software depois do problema; é reduzir a formação de hotspots por placement e migração.

15. Metodologia de validação

O projeto precisa passar por uma sequência que vá da matemática ao FPGA/ASIC antes de qualquer alegação de superioridade.

1. Validação analítica: provar as fórmulas de topologia e as regras de coerência/roteamento.
2. Simulação de grafos: comparar grade, honeycomb, HexaMesh e variantes H6 para N crescente.
3. Simulador NoC: medir hops, congestionamento, latência, throughput e deadlock.
4. Simulador de memória: medir hits, bytes transferidos, prefetch accuracy, pollution e compressão.
5. Co-simulação HARC: executar o controlador sobre traces reais de kernels.
6. RTL: implementar roteador, H-MESI e primitivas H6-ISA em Verilog/SystemVerilog.
7. FPGA: validar protocolo, ISA, runtime e segurança.
8. Síntese ASIC: estimar área, timing e energia.
9. Benchmark final: comparar com CPU, GPU e aceleradores especializados em workloads equivalentes.

15.1. Critérios de sucesso

| Métrica | Meta inicial | Critério |
|--------------------|-----------------------------|---------------------|
| Deadlock | 0 em campanha de teste | prova + stress |
| Coerência | 0 violações | litmus + randomized |
| HARC safety | 0 ações fora dos limites | formal/runtime |
| Memory utilization | < 0,95 alvo de referência | simulação |
| NoC throughput | > grade baseline | simulação |
| Energy/op | < baseline no workload-alvo | RTL/síntese |
| Speedup | medir, não assumir | benchmark |

16. Roadmap de implementação

| Versão | Entrega | Principal risco |
|--------|--|----------------------------|
| v0.3 | especificação integrada deste whitepaper | consistência entre módulos |
| v0.4 | simulador de grafo + H6-NoC | modelo de tráfego |
| v0.5 | simulador H-MEM + H-MESI | coerência e locality |
| v0.6 | emulador H6-ISA | toolchain |
| v0.7 | HARC em trace replay | estabilidade do agente |
| v0.8 | RTL de router/coherence | timing/area |
| v0.9 | FPGA prototype | I/O e memória |
| v1.0 | chiplet reference design | ASIC/package |

16.1. Primeiro protótipo recomendado

O protótipo mais informativo é pequeno, mas contém todos os mecanismos: 3x3 ou 5x5 células lógicas H6, dois VCs, diretório H-MESI, L0/L1/L2, HARC simples e 8-12 instruções H6-ISA. A finalidade é observar o sistema completo em vez de maximizar escala antes de validar os protocolos.

17. Limites e questões abertas

A H-HPU já possui uma arquitetura consistente no nível conceitual, mas ainda existem questões de pesquisa que não podem ser resolvidas por argumento geométrico isolado.

- Qual representação hiperbólica oferece melhor compromisso entre precisão, hardware e compressão?
- Qual largura de palavra minimiza área e erro para HDIST/HLOGM/HEXPM?
- Quanto do ganho topológico sobrevive quando PHY, buffers, arbitragem e memória são incluídos?
- Qual algoritmo HARC produz menor overhead sob workloads não estacionários?
- Quando a replicação reduz latência e quando aumenta desnecessariamente a coerência?
- Qual arquitetura de HBM/L3 é suficiente para manter U_M abaixo de 0,95 em escala?
- Qual tamanho de chiplet maximiza yield sem aumentar demasiado a área e o custo dos links?
- Quais kernels realmente obtêm $O(N \log N)$ em vez de apenas menor constante?
- Como mapear operações hiperbólicas para unidades fixas, CORDIC, LUT, aproximadores ou floating point dedicado?

17.1. Gargalo físico residual

O modelo atual consegue reduzir os gargalos de computação, rede e memória para abaixo da saturação. Isso não prova que a memória deixará de ser o gargalo em escala maior. Quando N, largura de modelo ou contexto cresce, a capacidade física e a largura de banda da memória podem voltar a dominar. Trabalhos recentes sobre memória HBM/HBF para inferência de LLM reforçam que capacidade e tráfego de memória continuam sendo um problema arquitetural relevante. [R7]

18. Conclusão

A H-HPU é uma proposta de co-design em que a geometria não é um detalhe de implementação, mas parte da máquina. O hexagonal foi incorporado porque uma rede planar de grau médio até 6 oferece uma estrutura natural para seis vizinhos locais, e a literatura de HexaMesh demonstra ganhos topológicos e de interconexão sob hipóteses bem definidas. [R1]

A partir desse núcleo, o projeto constrói H6-NoC para comunicação, H-MEM para reduzir movimentação de dados, H-MESI para coerência direcional, HARC para adaptação e correção e H6-ISA para expor as primitivas ao software. A integração desses módulos produz uma plataforma na qual o sistema pode aprender com seu próprio estado sem transferir para a IA as garantias que pertencem ao hardware.

O passo decisivo não é adicionar mais componentes, mas medir a arquitetura como um sistema: mesmas tarefas, mesmos dados, mesma precisão e comparação contra baselines estabelecidos. O resultado final deve ser determinado por latência, throughput, energia por operação, tráfego de memória, robustez e custo físico.

$$\text{H-HPU} = \text{Matemática} + \text{Dados} + \text{Topologia} + \text{Memória} + \text{Controle} + \text{ISA} + \text{Software}$$

Essa igualdade resume a hipótese central do projeto: para uma nova classe de matemática computacional, a máquina mais eficiente pode exigir que a matemática, o formato dos dados e a arquitetura sejam desenhados juntos.

Apêndice A. Notação

| Símbolo | Significado |
|---------|------------------------------------|
| N | número de unidades lógicas na rede |
| V | conjunto de vértices |
| E | conjunto de links |
| D | diâmetro da rede |
| B | largura de bisseção |
| d_avg | grau médio |
| R | relação entre unidades |
| N(v) | vizinhança do vértice v |
| T | tempo total |
| E | energia |
| U | utilização demanda/capacidade |
| BW | largura de banda |
| H | número de hops |
| C_place | custo de colocação |
| HARC | controlador adaptativo |
| H-MESI | protocolo de coerência |
| H6-ISA | extensão de instruções H6 |

Apêndice A.1. Equações centrais

$$\begin{aligned}
 E &\leq 3N - 6 \\
 d_{avg} &\leq 6 - 12/N \\
 D_G &= 2 \cdot \sqrt{N} - 2 \\
 D_H &= (1/3) \cdot \sqrt{12N - 3} - 1/2 \\
 B_G &= \sqrt{N} \\
 B_H &= (2/3) \cdot \sqrt{12N - 3} \\
 D_H/D_G &\rightarrow 1/\sqrt{3} \\
 B_H/B_G &\rightarrow 4/\sqrt{3} \\
 r_{min} &= 1 - 1/U_{M,overhead} \\
 U_{M,10\%} &= 1,03356 \cdot (0,90) = 0,93020
 \end{aligned}$$

Apêndice B. ISA de referência

A tabela a seguir é uma especificação funcional de software/hardware. Os campos de codificação precisam ser congelados somente após definir a microarquitetura.

| Instr | Grupo | Entradas | Saída/efeito |
|--------|-------|------------------|----------------|
| HLOAD | MEM | desc | tile local |
| HSTORE | MEM | tile, desc | memória |
| HPREF | MEM | stream, horizon | prefetch |
| HENC | MEM | parent, delta | residual |
| HDEC | MEM | parent, residual | estado |
| HROUTE | NOC | tile, q, r | tx id |
| HWAIT | NOC | tx id | status |
| HACQ | COH | addr | token |
| HREL | COH | addr | release |
| HINV | COH | addr | invalidate |
| HACK | COH | token | ack |
| HDIST | GEO | x, y | distance |
| HINNER | GEO | x, y | inner product |
| HLOGM | GEO | base, x | tangent vector |
| HEXPM | GEO | base, v | point |
| HFAULT | SYS | none | status |

| Instr | Grupo | Entradas | Saida/efeito |
|-------|-------|----------|--------------|
| HCRC | SYS | block | check status |

Apêndice C. Pseudocódigo

C.1. Roteamento H6

```
function route(src, dst, telemetry):
    candidates = minimal_paths(src, dst)
    legal = filter_turn_rules(candidates)
    legal = filter_faults(legal)
    if legal is empty:
        legal = controlled_bypass(src, dst)
    return argmin(legal,
        hops + lambda_q*congestion + lambda_e*energy + lambda_f*fault_risk)
```

C.2. H-MESI

```
function acquire(addr, requester):
    dir = home_directory(addr)
    lock(dir, addr)
    sharers = dir.sharers(addr)
    invalidate(set_difference(sharers, {requester}))
    wait_for_acks(set_difference(sharers, {requester}))
    version = dir.version(addr) + 1
    dir.owner(addr) = requester
    dir.version(addr) = version
    state(addr) = M
    unlock(dir, addr)
    return token(version)
```

C.3. HARC

```
loop:
    s = read_telemetry()
    features = normalize(s)
    proposal = policy.predict(features)
    action = safety_filter(proposal)
    apply(action)
    log(s, proposal, action)
```

Referências

[R1] P. Iff, M. Besta, M. Cavalcante, T. Fischer, L. Benini, T. Hoefler. "HexaMesh: Scaling to Hundreds of Chiplets with an Optimized Chiplet Arrangement." arXiv:2211.13989, 2022. Resultados reportados no trabalho: diâmetro -42%, bisseção +130%, latência -19%, throughput +34% sob seus modelos e simulações.

<https://arxiv.org/abs/2211.13989>

[R2] Z. Diao, R. Sun. "Minimal Deadlock-Free Routing for Degree-Six Triangular-Lattice Meshes and Tori with Two Forbidden Turns." arXiv:2609.09746, submetido em 9 Sep 2026. Construções de uma VC para malha finita, duas VCs para toro e bypass sob falha estática.

<https://arxiv.org/abs/2609.09746>

[R3] RISC-V International. Ratified Specifications Library. Versão da ISA não privilegiada v20260120. A especificação mantém espaços custom-0 a custom-3 para extensões customizadas.

<https://docs.riscv.org/reference/isa/unpriv/rv-32-64g.html>

[R4] NVIDIA. "How to Write High-Performance Matrix Multiply in NVIDIA CUDA Tile." 14 Jan 2026. Descrição de tiled matrix multiplication e uso de Tensor Cores via `ct.mma()`.

<https://developer.nvidia.com/blog/how-to-write-high-performance-matrix-multiply-in-nvidia-cuda-tile/>

[R5] International Energy Agency. "Energy and AI" / "Energy demand from AI." 2025. Estimativas de 415 TWh de consumo de data centers em 2024 e 945 TWh em 2030 no caso-base.

<https://www.iea.org/reports/energy-and-ai/energy-demand-from-ai>

[R6] R. Bera, R. Nadig, O. Mutlu. "Machine Learning-Driven Intelligent Memory System Design: From On-Chip Caches to Storage." arXiv:2603.14583, 2026. Prefetch, placement e políticas adaptativas guiadas por ML.

<https://arxiv.org/abs/2603.14583>

[R7] "HBM-HBF-Centric Memory Pooling Architecture With Custom Base Die for Terabyte-Scale LLM Inference." IEEE Computer Architecture Letters, 2026. Exemplo recente de arquitetura de memória para lidar com footprint de pesos e KV cache além da capacidade HBM.

<https://ieeexplore.ieee.org/document/11568525>

[R8] N. He, M. Yang, R. Ying. "HyperCore: The Core Framework for Building Hyperbolic Foundation Models with Comprehensive Modules." arXiv:2504.08912, 2025. Framework para modelos hiperbólicos e módulos de fundação.

<https://arxiv.org/abs/2504.08912>

[R9] "Hyperbolic Deep Learning System for Intelligent Gate-Driving and Health Monitoring of Silicon-Carbide Power MOSFETs." IEEE Open Journal of the Computer Society, 2025. Exemplo de pipeline de deep learning com camadas hiperbólicas e mecanismos especializados.

<https://ieeexplore.ieee.org/document/11177160>

[R10] "Hyperbolic CORDIC-Based Architecture for Computing Logarithm and Its Implementation." IEEE. Exemplo de implementação de primitivas hiperbólicas em hardware.

<https://ieeexplore.ieee.org/document/8985267>